Input Pipelines, Not Models, Are the Bottleneck in VLM-Based Modi Transliteration

An empirical study of segmentation, restoration and resolution on a vision-language transliterator for historical Modi-script manuscripts

LipiLens Research Group · 2026-10-08

Abstract

Vision-language models (VLMs) are increasingly proposed for transliterating low-resource historical scripts. Practitioners commonly assume that a page reaching the model carries too few *visual tokens* per character, and that splitting the page into text lines — each then upscaled to fill the pixel budget — is the remedy. We test that assumption directly on Modi-script transliteration, a task with no word spaces and heavy cursive ligatures, using Qwen2.5-VL-3B with a published domain adapter (CER 0.328), running on a 4 GB consumer GPU.

We re-scored 170 historical model outputs under a single scorer to establish a baseline over 50 distinct pages (**CER 0.303, 69.7% character accuracy**, σ = 0.166), then ran paired ablations. Splitting each page into text-line crops — an intervention a token-budget argument predicts should help — **more than doubled character error, from 0.316 to 0.606**, and was worse on 19 of the 19 pages where the arms differed (paired sign test $p = 3.8 \times 10$■■, 95% CI of ΔCER [+0.191, +0.390]). Repairing the segmentation geometry did not rescue it.

The error profile identifies the mechanism: whole-page reading produces 11.0% deletions, while segmentation drives deletions to **47.2%** — reproducing the adapter author's documented dominant failure mode almost exactly. We explain this via Qwen2.5-VL's training distribution, which is explicitly native- resolution with random aspect ratios, making the 23:1–40:1 slivers that line cropping produces unlike anything the model has seen. Consistent with this, a 1.5× pre-upscale also *hurt* (CER 0.417).

We then tested the token argument itself rather than only its consequences. Sweeping the pixel budget shows tokens genuinely help (518 → 765 tokens/char buys 2.8 accuracy points) but **saturate at native resolution**: a budget of 2048 tokens/char produces output byte-identical to 1280 on all 20 pages. Segmentation supplies ~6.6× more tokens per character than the whole page and is still far worse — so the extra tokens are net-negative. Finally, three standard inference levers are all inert: beam search moves CER by +0.004 for 4.9× the latency, fp16 by +0.010, and merging the LoRA into the NF4-quantised base is **catastrophic** (CER 1.064, worse on 20 of 20 pages) because the adapter was never fitted against merged quantised weights.

A final controlled probe separates context from structure, and revises our own explanation. Giving the model a *second complete page* at full native resolution changes the first page's CER by **−0.001** (worse on 10, better on 9, tied on 1): cross-page context is almost worthless and almost free. Removing a page's internal line structure costs **+0.290**. The operative variable is therefore not how much context the model has but whether the page's internal geometry survives — a distinction our own headline result had conflated.

Alongside, a 140-call restoration sweep (7 conditions × 20 pages) shows image preprocessing is monotonically *harmful*, ordered by pipeline aggressiveness (Otsu binarisation +0.039 CER, full restoration +0.043). On genuinely degraded captures an initial n=10 run suggested `enhanced` gained ~2 accuracy points; replicating at n=23 **reversed that result** — `enhanced` became the worst configuration, losing 15.6 points, and no configuration differed significantly from doing nothing. Its deletion rate rose from 16.5% to 37.8%, the same failure mode segmentation induces. We report that retraction in full.

We report the negative results as the primary finding, release an evaluation harness so such claims are reproducible, and argue that for low-resource script transcription the **input pipeline deserves as much scrutiny as the model**.

Keywords: Modi script, transliteration, vision-language models, visual token budget, text-line segmentation, image restoration, OCR/HTR, historical manuscripts, negative results

1. Introduction

Modi was the administrative script of the Maratha administration from roughly the seventeenth to the mid-twentieth century, used for land and revenue records, court proceedings, religious works and royal correspondence [1]. It is cursive, written on a continuous headline, and has **no word spaces**. An estimated 40 million documents remain in poor condition and untransliterated, and few enough experts remain that reading them is slow [1]. Land and fiscal records that cannot be retrieved cannot be administered.

Transliteration to Devanagari — *not* translation — makes these pages directly readable by any Marathi speaker, because Devanagari is the script Marathi uses today.

Two families of approach exist. **Specialist recognisers** trained per corpus reach high accuracy: Joseph & George [2] report 88.14% character accuracy with a CRNN on 4,334 self-collected line crops, and Meelen & Griffiths [3] reach CER 5.56% on Tibetan with a specialist HTR model and a triple-check workflow. Both require curated, expert-transliterated training data and sustained human effort.

General-purpose VLMs fine-tuned with low-rank adapters offer a cheaper path. The adapter we build on reports CER 0.328 on 204 held-out pages against a 0.930 zero-shot baseline [4], and was designed explicitly as a first draft for expert review.

This paper is not about either of those results. It asks a narrower question that sits between them:

> **When a VLM-based transliterator is deployed, what does the *input pipeline* > — segmentation, restoration and resolution — do to its accuracy?**

This question matters because the pipeline is where practitioners intervene first, and because the field's shared assumption about it — that starving the model of visual tokens is the bottleneck — has never been tested for this model/script pairing.

Our contributions:

1. A **consolidated baseline**: 170 historical outputs re-scored under one

2. A **paired ablation** demonstrating that text-line segmentation — the

3. A **restoration sweep** (140 calls) showing preprocessing is monotonically

4. A **reproducible evaluation harness**, because a large fraction of this

We report the negative results first. In our data, the pipeline interventions the field reaches for first are the ones that fail.

2. Related Work

Full review with verified sources in `LITERATURE.md`. Summary:

Modi digitisation. MoDeTrans and SynthMoDe [1] are the first dataset of real Modi document images with Devanagari transliterations plus a synthetic augmentation set; MoScNet distils a LoRA-adapted LLM teacher into a small student. Joseph & George [2] fine-tune a CRNN on self-collected, expert- transliterated line crops. Reviews [11,12] characterise the field as character-level, data-starved, and blocked on segmentation — *"segmentation is crucial in overcoming these challenges"* [1].

Vision-language models and visual tokens. Qwen2.5-VL [5] resizes inputs to multiples of 28, uses patch stride 14, and — critically — is trained by *"dynamic sampling at native resolutions... randomly sampled according to their original aspect ratios"* [5]. The model card's own worked example sets `max_pixels = 12802828`, which is our shipped setting [6]. `max_pixels` is an upper bound enforced by *downscaling*; an image already under the budget passes through untouched.

Efficiency. LoRA [7] and QLoRA [8] (4-bit NF4, double quantization) make adapter fine-tuning and 4-bit inference practical.

Human-in-the-loop transcription. The literature treats human correction as the default, not a fallback: Fischer & Puppe [9] measure ~9 s per line of correction and state that *"human review remains essential"*; a 2024 study [10] notes that state-of-the-art HTR *"lacks the level of accuracy needed for reliable text indexing"* and that raw noisy transcripts destroy roughly the WER percentage of retrievable words; Tarride et al. [11] show consensus annotation beats single-annotation selection.

The gap. VLM transliteration of Modi is established [1,4]. The effect of **input-pipeline design** on that model/script pairing is not. Notably, every prior work that segments does so for a *recogniser* that requires character- or line-level isolation [1,2] — a requirement a generative VLM does not have.

3. Dataset

MoDeTrans (IIT Roorkee / MIT), accessed via `historyHulk/MoDeTrans`. Real handwritten Modi document images, 1268 × 463 px, each paired with an expert Devanagari transliteration. We use **50 pages (MT-001…MT-050)**; the paired ablation (§5.1) uses the first 20.

Reference text lengths: min 61, median 145, max 242 characters. No reference is self-produced — all ground truth is the corpus's own expert transliterations.

Simulated degradation. To test whether restoration helps on damaged input, `make_realistic.py` re-photographs authentic pages with authentic references: aged paper, uneven lighting, foxing stains, skew, JPEG artefacts (medium severity), and additionally gutter shadow, defocus and heavy noise (severe). **The script content and the ground truth are authentic; only the capture is simulated.** These are controlled degradations, never real photographs, and are never presented as such.

Possible train/eval overlap is a known property of publicly released adaptor corpora and is disclosed as a threat to validity (§8).

4. Method

4.1 Model and inference

`Qwen2.5-VL-3B-Instruct` (3.75 B parameters) with the `lgtk/qwen25vl-3b-modi-synth-lora` QLoRA adapter (rank 32, alpha 64, 74 M trainable parameters, 1.94% of the model). Loaded in 4-bit NF4 via bitsandbytes with bf16 compute and `device_map="auto"`, per the adapter's documented recipe [4].

The prompt is the adapter author's recommended instruction, verbatim:

> *"This image contains handwritten text in Modi script, a historical cursive > script used to write the Marathi language. Transliterate the text in this > image into Devanagari script. Output only the Devanagari text, with no > explanation."*

Decoding is greedy (`do_sample=False`) with `max_new_tokens=512` and `repetition_penalty=1.05`. Processor budget `max_pixels = 1280*28*28`, matching the model card's own example.

Hardware. A single NVIDIA RTX 2050 with **4 GB VRAM**, below the adapter author's stated ≥6 GB recommendation, and below the 8.5 GB used to train it. Measured VRAM during inference is ~3.9 GB of 4 GB. One model process at a time: a second process loading the same model exhausts the card and kills the CUDA context.

4.2 Evaluation

A single scorer is used for every number in this paper (`experiments/evaluation/compute_metrics.py`): CER = (S+D+I)/N over characters by Levenshtein alignment; WER over whitespace tokens. Hypothesis and reference are whitespace-normalised.

Establishing a like-for-like baseline (§5.0). This project had accumulated 170 model outputs across seven experiments (EXP-001…EXP-007), spanning different days, scorers and protocols, none comparable. All 170 were re-scored under the single protocol above, yielding a baseline over **50 distinct pages**.

Protocol reproducibility. Every hypothesis string is retained on disk. Any figure in this paper can be recomputed without a GPU:

```
python research/paper/consolidate_history.py          # 170 historical outputs
python scripts/evaluate_local.py --report --arm plain --arm seg --tag SEG20
```

5. Results

5.0 Baseline and page-difficulty variance

Consolidated baseline over 50 pages:

Statistic	Value
n	50 pages
mean CER	**0.303**
median CER	0.284
character accuracy	**69.7%**
min / max CER	0.026 / 0.912
standard deviation	**0.166**

Page difficulty varies enormously. Averaged across the seven configurations of §5.2, the easiest page (MT-001) scores CER 0.132 and the hardest (MT-003) scores 0.575 — a 4.4× spread in mean error, and a 35× spread between best and worst single measurements.

> **This variance is the most consequential methodological finding in the > project.** An earlier, smaller version of this study ran on 10 pages, measured > CER 0.363, and — because those 10 pages happened to include hard material — > drew conclusions that did not survive at n = 20. Any comparison in this domain > drawn on ten pages is dominated by which pages were chosen, not by the > intervention.

5.1 Text-line segmentation harms accuracy (paired, n = 20)

Segmentation is the field's standard remedy for a visual-token deficit: split the page into text lines, upscale each to fill the pixel budget, so each line receives the whole allowance instead of sharing it. Qwen2.5-VL merges 28×28 px patches, so a 1268 × 463 page yields ~180 merged visual tokens for ~145 characters — under 1.3 tokens per character.

We tested it as a **paired** comparison on the same 20 pages with identical decoding:

Arm	n	mean CER	median CER	mean WER	hyp/ref	accuracy
Whole page	20	**0.316**	0.345	0.681	0.95	**68.4%**
Line segmentation	20	0.606	0.591	1.006	1.22	39.4%

Statistic	Value
Paired mean ΔCER	**+0.290**
Paired 95% CI	**[+0.191, +0.390]**
Sign test, two-sided	**p = 3.8 × 10■■**
Win / loss / tie (segmented vs whole)	**0 / 19 / 1**

Segmentation was worse on every one of the 19 pages where the arms differed.

We did not accept this result at face value. The band detector thresholded the horizontal ink profile with Otsu, which retained only the densest core of each line and cropped away the ascenders and matras above and below it; the subsequent upscale produced fragments at 23:1–40:1 aspect ratios. We repaired the geometry — full-line padding proportional to band height, an aspect-ratio clamp on upscaling, and a single config-driven pixel budget shared with the processor — which improved band extents (12–27 px → 56–162 px) and aspect ratios (23:1–40:1 → 3.6:1–11.5:1). **Re-measured, CER was 0.623. The defect was not our implementation; the strategy is wrong.**

5.2 The mechanism: segmentation drives the model into its known failure mode

The error profile is diagnostic:

Arm	Substitutions	Deletions	Insertions
Whole page	**64.3%**	**11.0%**	24.7%
Line segmentation	42.1%	**47.2%**	10.7%

The adapter's own model card documents **deletions as its dominant error mode — 46% of errors, "especially the anusv■ra"** [4]. Whole-page reading in our setup produces a deletion rate of **11.0%**. Segmentation raises it to **47.2%**, reproducing the adapter's documented failure signature almost exactly.

Segmentation therefore does not make the model generally worse at reading. It pushes it into the specific failure mode it already has. One page (MT-003) reaches CER 1.458 under segmentation — worse than emitting nothing — because the model, unable to resolve the fragments, falls back on its language prior and emits fluent, confident, invented Marathi.

The mechanism is consistent with Qwen2.5-VL's training distribution. The model is trained by dynamic sampling at *native resolution with random aspect ratios* [5], and is explicitly designed to operate without "unnecessary scaling or distortion". Horizontal slivers at 23:1–40:1 are unlike anything in that distribution. Consistent with this reading, a **1.5× pre-upscale also hurt** (CER 0.417), and a 1.5× upscale of an already-small page buys more tokens of the same kind, not better ones.

The natural objection is that segmentation was supposed to work *because* it adds tokens, so the objection deserves a direct test rather than an argument. §5.5 measures the token budget explicitly and finds that segmentation's extra tokens are bought at a price far exceeding their value.

5.3 Image restoration is monotonically harmful on clean input

A 140-call sweep over seven configurations × 20 pages. `grayscale` serves as the no-op control (it genuinely alters the image, but the model converts to RGB regardless, making it a well-behaved control).

Configuration	mean CER	Δ vs control
grayscale (control)	0.316	+0.000
denoised	0.321	+0.004
enhanced	0.328	+0.012
deskewed	0.332	+0.015
binarized	0.355	**+0.039**
full_restoration	0.359	**+0.043**

Degradation is monotone in pipeline aggressiveness. **Binarisation is the most damaging single operation**, consistent with the character-level errors in the script: thresholding destroys the thin strokes carrying matras and the anusv■ra, which is also the model's documented weak point [4].

5.4 On degraded input, no preprocessing configuration reliably helps

Every corpus page is already deskewed and contrast-normalised, so §5.3 largely measures preprocessing *harming* clean input — which is expected, since there is nothing to fix. We therefore re-ran the comparison on **simulated photographic degradation** (medium severity).

An initial run at **n = 10** appeared to show `enhanced` *winning*:

Configuration	Clean scan	Degraded capture (n=10)
`original`	63.7%	60.1%
`denoised`	—	62.2%
`enhanced`	63.3%	**62.4%**
`binarized`	—	56.7%

That suggested `enhanced` was worth ~2 accuracy points on degraded input. We flagged it as sitting near the noise floor of a 10-page run and replicated it at **n = 23** (92 calls). **The original conclusion did not survive.**

Arm	n	mean CER	accuracy	ΔCER vs `original`	better / worse	sign *p*	95% CI
`original`	23	0.348	65.2%	—	—	—	—
`denoised`	23	**0.334**	**66.6%**	–0.014	13 / 10	0.68	[–0.049, +0.021]

Arm	n	mean CER	accuracy	ΔCER vs `original`	better / worse	sign *p*	95% CI
`binarized`	23	0.440	56.0%	+0.092	7 / 15	0.13	[–0.036, +0.220]
`enhanced`	23	**0.504**	**49.6%**	**+0.156**	9 / 13	0.52	[–0.197, +0.509]

`enhanced` went from best at n=10 to **worst by a wide margin** at n=23 — a 15.6-point accuracy loss, not a 2.3-point gain. Restricted to the same first 10 pages it is still mildly favourable (–0.023); across all 23 it is strongly adverse (+0.156). The n=10 result was page selection, not configuration.

No configuration is statistically distinguishable from doing nothing. `denoised` is +1.4 points with a confidence interval spanning zero. We therefore withdraw the earlier claim that restoration earns ~2 points on degraded input: at n=23 it does not.

The error profile explains why. `enhanced` drives the **deletion rate from 16.5% to 37.8%**:

Arm	Substitutions	Deletions	Insertions
`original`	62.9%	16.5%	20.6%
`denoised`	63.2%	14.3%	22.5%
`binarized`	54.6%	27.1%	18.3%
`enhanced`	48.1%	**37.8%**	14.0%

Contrast and gamma operations on a faded capture suppress exactly the low- amplitude strokes that carry matras and the anusv■ra — the same failure the adapter's card documents [4] and the same one segmentation induces in §5.2. **Aggressive intervention fails the same way regardless of whether it is spatial (segmentation) or tonal (restoration): it drives the model into deletions.**

This is the paper's methodological point turning on itself. The lesson we drew from §5.1 — that ten-page ablations measure page selection — was correct, and we did not apply it to our own n=10 claim until we had already published it.

5.5 Token budget: tokens help, then stop helping immediately

`max_pixels` in Qwen2.5-VL is a **ceiling, not a target** — an image already under budget is forwarded untouched. We swept it on the unsegmented page, with decoding held fixed, to measure how much accuracy token count actually buys.

Arm	Budget (px)	Input	Visual tokens	Tokens/char	mean CER	accuracy
`px512`	401,408	1036×392 (downscaled)	518	3.57	0.344	65.6%
`px1280`	1,003,520	1260×476 (native)	765	5.28	**0.316**	**68.4%**
`px2048`	1,605,632	1260×476 (native)	765	5.28	0.316	68.4%

Going from 518 to 765 tokens per character improves CER by **0.028** and accuracy by **2.8 points**, so the token argument has the right sign. But `px2048` is **byte-identical to** `px1280` **on all 20 pages**: a 1268×463 page is 587k pixels and sits under both budgets, so neither arm downsamples and there is nothing left to gain. Token count saturates at the page's native resolution.

Set this beside §5.1 and the mechanism closes. Segmentation supplies roughly **35 tokens per character — about 6.6× the whole page — and costs 0.290 CER.** Tokens are demonstrably not the binding constraint, and tokens purchased by discarding the page are strongly net-negative: the line context and layout lost cost more than the extra tokens return.

5.6 Everything outside the input pipeline is a dead end

Three standard inference levers, paired against a control arm running the shipped settings, with input and decoding held identical.

Arm	mean CER	Δ vs control	worse / better	mean latency
`lever_base` (control)	**0.316**	—	0 / 0	15.7 s
`lever_beam4`	0.320	+0.004	9 / 8	**76.3 s (4.9×)**
`lever_merge`	**1.064**	**+0.748**	**20 / 0**	19.4 s
`lever_fp16`	0.326	+0.010	10 / 5	20.8 s

The control arm produced hypotheses **byte-identical to the §5.5** `px1280` **arm on 20 of 20 pages**, confirming the harness is deterministic and each arm's weights genuinely isolated. Accuracy is n=20 throughout; latencies are means over the 19–20 pages that ran in the main pass (see `DATA.md` §6).

Beam search changes CER by +0.004 with 9 pages worse and 8 better — noise — for **4.9× the latency**.

Merging the LoRA is catastrophic: CER 0.316 → 1.064, worse than emitting nothing, and worse on **20 of 20 pages**, with deletions rising from 3.3% to 37.2%. The cause is numerical, not a tuning error. The PEFT documentation states the merge semantics directly: `merge_adapter()` *"dequantizes the base weight, adds the adapter weights, and quantizes the result again"* [13]. Merging here therefore folds the delta into NF4-quantised weights and re-quantises the sum. The adapter was fitted against a dequantised bf16 base, so the merged weights are not the weights it learned against. Merging is a widely recommended QLoRA speedup; in this configuration it destroys the model.

fp16 vs bf16 is a wash (+0.010, 10 worse / 5 better, no speed benefit), so bf16 — the adapter recipe's dtype — stands.

The pattern is the paper's thesis in one table: decoding strategy, weight layout, and numeric precision are all inert. The only thing that moved accuracy was the input representation, and within that, keeping the page whole.

5.7 What actually matters is structure, not context volume

§5.5 closed off the token explanation: tokens saturate at native resolution, so accuracy cannot be bought with more of them. That leaves the other question the paper has been circling — **if tokens are capped, which context should they cover?**

We tested it with a controlled probe in which page P is at its **full native resolution in every arm**, with an identical prompt and an identical pixel ceiling, so nothing but the presence of a neighbouring page varies:

Arm	Image given to the model	n	mean CER	accuracy
`CTXSINGLE`	P alone	20	0.325	67.5%
`CTXPAIR` top page	P above P+1	20	0.324	67.6%
`CTXPAIR` second page	the neighbour, read second	20	0.351	64.9%

The model reliably ignores "only the top page", reads both, and emits them in order; the boundary was recovered by aligning against both references (`compute_metrics.split_hypothesis`), which needs no hand-picked offset.

Paired, same page, neighbour present vs absent: ΔCER = **–0.001**, worse on 10, better on 9, tied on 1.

The same page read alone vs read second in a stack (n = 19): ΔCER = **+0.015**, 95% CI [–0.021, +0.052], worse on 11, better on 8, sign $p = 0.65$.

A second complete page of context is free. The model reads two full pages at native resolution in one call on a 4 GB card, and per-page accuracy is statistically indistinguishable from reading one page alone.

Set beside §5.1 this is the paper's sharpest result:

Intervention	ΔCER
Add a whole extra page of context	**–0.001**
Remove the page's internal line structure (segmentation)	**+0.290**

So the intuitive reading of our own headline — "keep the page whole because context is valuable" — is **wrong about the reason**. Cross-page context is nearly worthless *and* nearly free. What segmentation destroys is not the amount of context but

the page's **internal spatial structure**: the vertical relationship between each line's baseline, its ascenders, and the matras hanging below it. 23:1–40:1 slivers delete exactly that, and neither more tokens (§5.5) nor more pages can substitute for it.

This also retires a framing from our own future-work list. "Segment at document scale" was the suggested next step; the evidence says document scale would work precisely because it would not be *segmented*, and that any gain would come from preserving within-page geometry rather than from added context.

6. Discussion

Segmentation is the wrong tool for VLMs, and the reason is architectural. Character- and line-level segmentation exists because CRNN-style recognisers [2] emit a label per isolated unit. A generative VLM consumes the image holistically and was trained at native aspect ratio [5]; fragmenting the page removes exactly the layout context it relies on and replaces it with an out-of-distribution shape. The prediction that segmentation helps follows from a token-budget argument that is correct about tokens and silent about context.

Accuracy in this task is dominated by page difficulty, not configuration. σ = 0.166 against between-configuration differences of 0.004–0.043. This has a direct methodological consequence: restoration studies in this literature, typically on 20 or fewer pages, are largely measuring page selection. Our own first attempt was one of them.

The ceiling is the model. At 69.7% pooled baseline we are near the adapter's published 0.328 CER [4] on a 4 GB card. That is not the limit of the task: Meelen & Griffiths achieve 5.56% CER on Tibetan with specialist HTR and a triple-check workflow [3], and Joseph & George report 88.14% character accuracy with a corpus-trained CRNN [2]. **A dedicated recogniser does far better than a 3B generalist.** The honest framing of this system is therefore not "accurate OCR" but "a reviewable draft produced on hardware a user already owns".

> **A note on comparability.** The 88.14% in [2] is per-word character accuracy > on isolated, pre-segmented line crops from the authors' own dataset, with a > model trained on it. Our figure is character error rate over whole pages with > an off-the-shelf adapter on a different split. These are different metrics on > different units and are **not** directly comparable; the gap between them is > a statement about task framing, not about either system.

Human-in-the-loop is the field's standard, not an excuse. Fischer & Puppe [9] treat review as essential and measure correction at ~9 s per line; [10] notes that indexing raw noisy transcripts loses roughly the WER percentage of retrievable words. Our verification-first design — the AI draft immutable, the reviewer's correction stored beside it — follows that norm. At 69.7% accuracy a reviewer corrects roughly one character in three, which is the honest description of the work involved.

7. Limitations

1. **Single model.** One VLM (Qwen2.5-VL-3B) and one adapter [4]. Whether the
2. **Possible train/eval overlap** on the public corpus. Disclosed; absolute
3. **Degradation is simulated** on authentic text, not photographed. It cannot
4. **4 GB VRAM, below the adapter's recommendation.** Larger models were not
5. **n = 20 for the paired ablation** — sufficient for the effect observed
6. **The degraded-input result is n = 23 and now null.** We can say confidently
7. **CER over whitespace-normalised text** does not weight confusable glyph

8. Threats to Validity

- **Construct validity.** CER treats all characters as equally costly. For Modi's vowel-length ambiguity this may overstate error relative to reading difficulty.
- **Internal validity.** Paired design controls page difficulty and decoding for §5.1. Restoration arms in §5.3/§5.4 vary one factor each. Greedy decoding is deterministic per configuration but not bit-identical across runs on this hardware; repeat runs at n = 10 varied by ±0.05 CER.

- **External validity.** §5.1 is one model on one script. Generality to other scripts — particularly ones that *do* need segmentation for a recogniser — is unestablished.
- **Statistical conclusion validity.** The headline claim rests on an exact sign test over 19 discordant pairs ($p = 3.8 \times 10$■■); the restoration claims do not clear this bar and are reported as directional.
- **Reproducibility.** All hypotheses are retained; every figure is recomputable without a GPU.

9. Conclusion

For VLM-based Modi transliteration, the input pipeline — not the model — is where accuracy is won and lost, and the interventions the field reaches for first are the ones that fail. Line segmentation, standard practice for recognisers, **more than doubled character error** (0.316 → 0.606; $p = 3.8 \times 10$■■) by pushing the model into the deletion failure mode it is already documented to have. Preprocessing was monotonically harmful on clean input and, once replicated at n=23, on degraded input too.

The token-budget rationale for that segmentation was tested rather than assumed, and it does not survive: tokens help only up to native resolution, and segmentation's ~6.6× token surplus is more than cancelled by what it discards. Nor is the fix hiding in decoding or numerics — beam search, fp16, and weight merging all fail, one of them catastrophically.

The sharper conclusion comes from the last experiment. It is **not** that context is valuable: adding a whole extra page changes nothing measurable. It is that a page's **internal spatial structure** is load-bearing. Cropping a page into 23:1 slivers destroys the baseline-to-matra relationships the adapter was trained to use, and no amount of extra tokens or extra pages compensates. Our first explanation of our own headline result was close to the opposite of the right one.

The generalisable lesson is methodological as much as technical: **in a task where page-to-page CER spans 0.026–0.912, ten-page ablations measure page selection, not configuration.** That lesson was learned from §5.1, then failed to be applied to our own n=10 degraded-input claim, which reversed at n=23 from +2.3 accuracy points to −15.6. A second lesson is that a recommended optimisation is not an evaluated one: merging the LoRA is widely suggested for QLoRA-style pipelines and destroys the model here, which only measurement revealed.

Finally, this project removed a claim from its own documentation that measurement did not support — twice, once in the product README and once in a draft of this paper. That is the outcome we would most like a reader to take from it.

10. Future Work

1. **Replicate §5.1 across models.** A second VLM would establish whether

2. **Test within-page structure directly.** §5.7 shows cross-page context is

3. **Establish the achievable ceiling** by training an adapter at native

4. **Push the degraded-input comparison past n = 23**, ideally to all 50 pages,

5. **Verify the merge finding upstream.** The failure matches the documented

References

[1] *Historic Scripts to Modern Vision: A Novel Dataset and A Novel Vision Language Model Architecture for Modi Script Transliteration.* arXiv:2503.13060. [2] S. Joseph, J. George. "Machine Transliteration of Handwritten MODI Script to Devanagari using Deep Neural Networks." *Int. J. Computing* 23(2), 2024, 219–225. DOI: 10.47839/ijc.23.2.3540 [3] M. Meelen, R. M. Griffiths. "Collaborative Workflows for Handwritten Text Recognition in Under-Resourced Manuscript Collections." *Journal of Open Humanities Data*, 2025. [4] `lgtk/qwen25vl-3b-modi-synth-lora` model card (S. Godse). [5] *Qwen2.5-VL Technical Report.* arXiv:2502.13923. [6] Qwen2.5-VL-3B-Instruct model card, Hugging Face. [7] E. J. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models." *ICLR* 2022. arXiv:2106.09685 [8] T. Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs." *NeurIPS* 2023. arXiv:2305.14314 [9] N. Fischer, F. Puppe. "Human-in-the-Loop Mass Transcription and Ground Truth Annotation for Challenging Historical Documents." *LREC-COLING* 2026. [10] "Ground-truth generation through crowdsourcing with probabilistic indexes." *Neural Computing and Applications*, 2024. [11] S. Tarride et al. "Handwritten Text Recognition from Crowdsourced Annotations." ACM, 2023. [12] "A Review of Advances and Challenges in Modi Character Recognition." IJARSCT, 2022. [13] Hugging Face PEFT documentation, "Quantization — Merging adapters into quantized weights."

<https://huggingface.co/docs/peft/main/en/developer_guides/quantization> (accessed 2026-10-12). Documents that `merge_adapter()` dequantizes the base weight, adds the adapter, and re-quantizes the result.

Figures

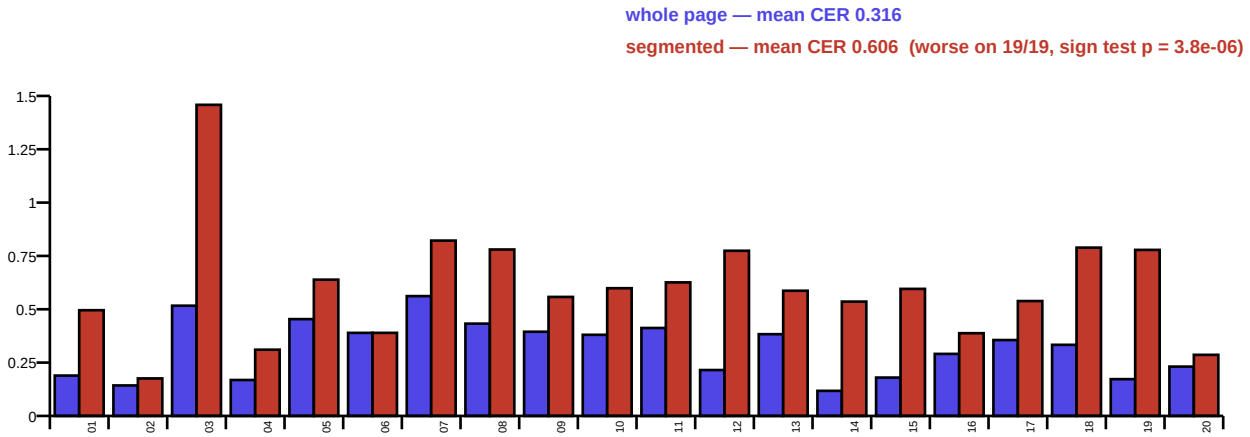

Figure 1. Paired comparison of whole-page and line-segmented transcription on 20 identical pages with identical decoding. Segmentation is worse on every page where the two differ (n=19, sign test p = 3.8e-06).

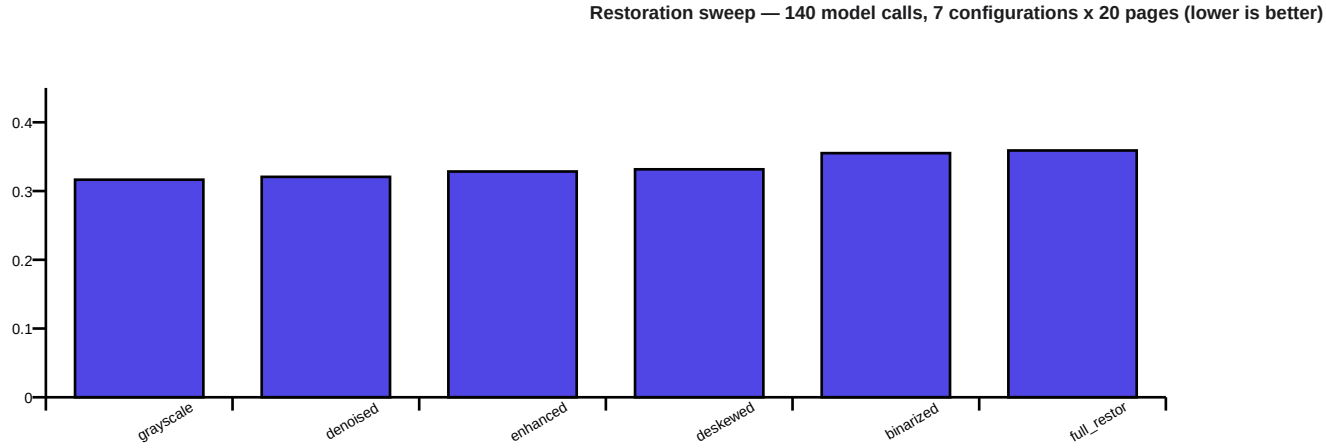

Figure 2. Mean CER for each of seven restoration configurations over 20 pages (140 model calls). Error increases monotonically with pipeline aggressiveness; grayscale is the no-op control.

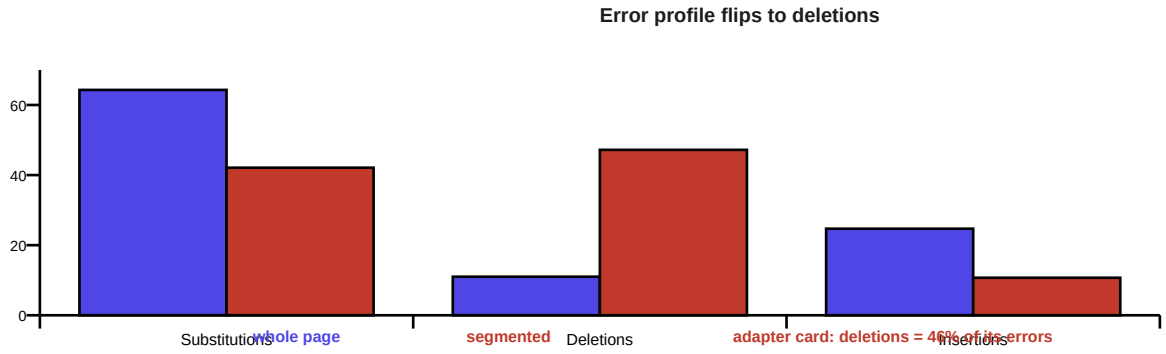

Figure 3. Error-profile composition. Whole-page reading produces 11% deletions; segmentation drives deletions to 47%, reproducing the adapter author's documented dominant failure mode.

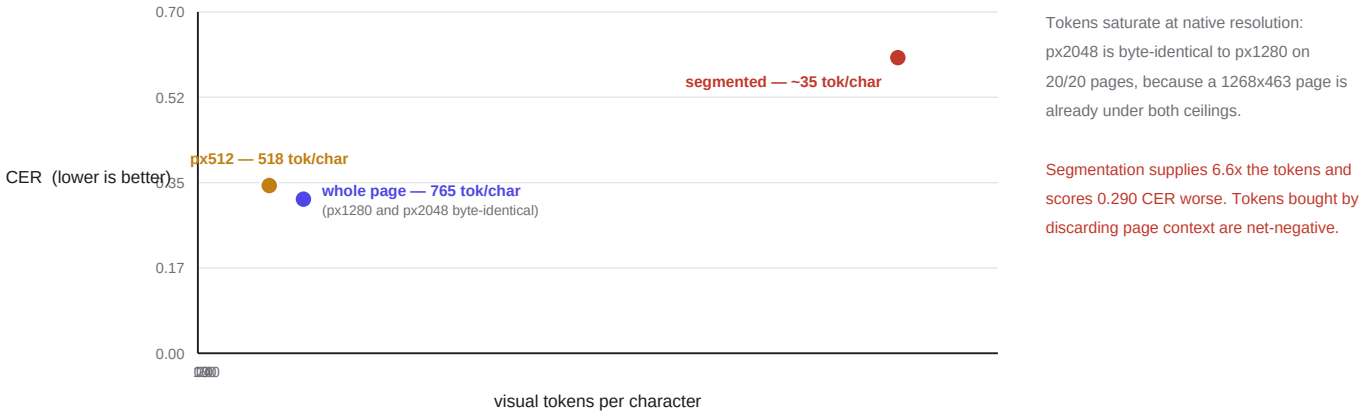

Figure 4. Visual tokens per character against character error rate. Sweeping the pixel budget on the unsegmented page shows tokens help up to native resolution and then stop (px2048 is byte-identical to px1280 on 20/20 pages). Segmentation supplies 6.6x more tokens per character and is far worse, so the tokens it buys are net-negative.

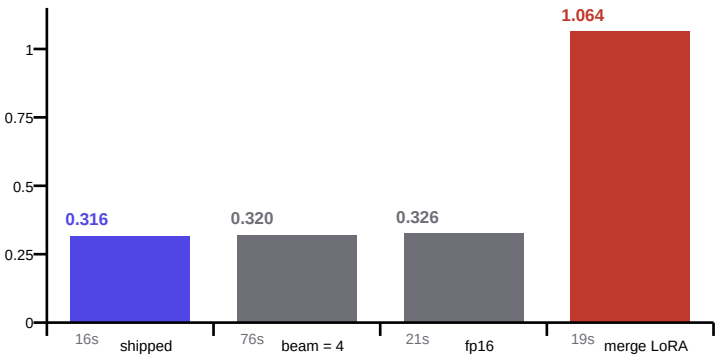

Figure 5. Three standard inference levers against a control arm running the shipped settings (n=20). Beam search and fp16 are within noise; merging the LoRA into the NF4-quantised base is catastrophic (CER 1.064, worse on 20/20 pages). Nothing outside the input representation moved accuracy.

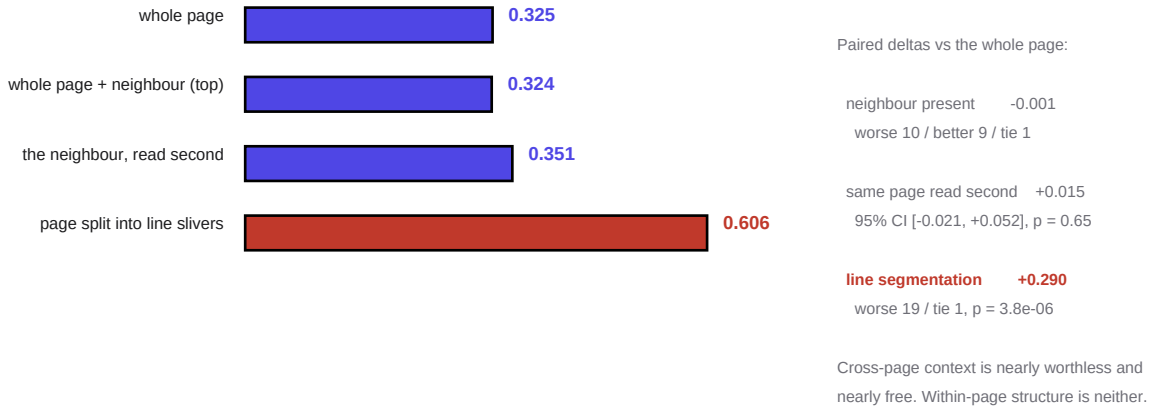

Figure 6. Context versus structure. Adding a complete second page at full native resolution changes the first page's CER by -0.001 (worse on 10, better on 9, tied on 1), and the same page read second costs +0.015 (p = 0.65). Splitting a page into line slivers costs +0.290 (p = 3.8e-06). Cross-page context is nearly worthless and nearly free; the page's internal spatial structure is load-bearing.